WebX

Course Code: ITC602

Module 2

TypeScript Classes and Objects

CLASS IN TYPESCRIPT

A class is a blueprint or template used to create objects. It defines properties (variables) and methods (functions).

Example

•Class → Student Form • Object → Filled Student Form

Example

```
class Person {
name: string; // Field

constructor(name: string) {
this.name = name;
}

printName(): void {
console.log("Name is: " + this.name);
}
}
```

Object (INSTANCE OF CLASS)

An object is a real instance created from a class.

Eg:

```
var obj1 = new Person("Tushar");
```

ACCESSING PROPERTIES AND METHOD

Access Field

```
console.log(obj1.name);
```

Access Method

```
obj1.printName();
```

INHERITANCE IN TYPESCRIPT

Inheritance allows one class to acquire the properties and methods of another class.

Parent and Child Class

• Parent Class → Base class • Child Class → Derived class

Keyword Used

extends

Example

```
class Shape {
perimeter: number;

constructor(p: number) {
this.perimeter = p;
}
}

class Square extends Shape {
printPerimeter(): void {
console.log("Perimeter of Square: " + this.perimeter);
}
}
```

Example

```
class Shape {
perimeter: number;

constructor(p: number) {
this.perimeter = p;
}
}

class Square extends Shape {
printPerimeter(): void {
console.log("Perimeter of Square: " + this.perimeter);
}
}
```

TYPES OF INHERITANCE

1. Single Inheritance
One parent → One child

2. Multilevel Inheritance
Class A → Class B → Class C

3. Multiple Inheritance
Not supported in TypeScript

METHOD OVERRIDING

Method overriding occurs when a child class provides a specific implementation of a method already defined in the parent class.

Keyword Used

super

Example

```
class A {
protected printSomething() {
console.log("Printing from Class A");
}
}

class B extends A {
protected printSomething() {
super.printSomething();
console.log("Printing from Class B");
}
}
```

ENCAPSULATION

Encapsulation is the process of hiding internal data and allowing access only through specific methods.

Access Modifiers:
public modifier can be used Anywhere

private modifier can be used only in the same class

protected modifier can be used in its class and child.

Example

```
public x: string = "Tushar";
private y: string = "Agarwal";
```

INTERFACES

An interface defines a structure or contract that a class or object must follow.

Why Interface is Needed

• Enforces consistency • Prevents structural errors • Improves readability

Example:

```
interface Student {
studentCode: number;
studentName: string;
}
```

IMPLEMENTING INTERFACE

```
class NewStudent implements Student {
studentCode: number;
studentName: string;
}
```

If any property is missing, compiler throws error.

TYPESCRIPT MODULES

A module is a file that contains code which can be reused in other files.

Why Modules?

· Avoid global variables · Improve security · Better organization

```
export class Student {}
import { Student } from "./Student";
```